인공지능수학 (2026-1)

4. TF-IDF 벡터를 활용한 텍스트 문서의 표현

국립금오공과대학교 컴퓨터공학부

컴퓨터공학전공 / 인공지능공학전공

김 경 수

목차

I. 텍스트 문서의 처리 절차

II. TF-IDF 벡터의 기본 개념

학습목표

① 텍스트 문서를 벡터로 표현하기 위한 전처리 작업의 필요성을 이해하고, 대표적인 전처리 작업의 종류를 열거하고 설명할 수 있다.

② 텍스트 문서를 벡터로 표현하기 위해 널리 사용되는 TF-IDF Score의 개념을 이해하고, 이를 위해 TF-Score와 IDF-Score를 계산하는 방법을 설명할 수 있다.

③ 실제 예제 문서가 주어졌을 때, 문서의 전처리와 전처리된 문서를 TF-IDF 벡터로 표현하는 일련의 전 과정을 스스로 수행할 수 있다.

Ⅰ. 텍스트 문서의 전처리 절차

텍스트 문서의 전처리(Preprocessing)란?

- 텍스트 문서 분석 시 의미에 큰 영향을 미치지 않는 불필요한 단어들을 제거하여 텍스트 문서 분석을 더욱 용이하게 함.
- 텍스트 문서 분석에 불필요하거나 방해가 되는 요소
  - 불필요한 단어
  - 변형된 단어
  - 기타
- 텍스트 문서의 전처리는 일련의 텍스트 시퀀스로 구성된 문서를 집합 또는 벡터로 수치화하여 표현하기 용이하도록 문서 내 모든 텍스트 데이터를 정제하는 작업을 의미한다.

텍스트 문서의 전처리 절차

- 기존의 텍스트 문서가 입력으로 주어지면, 아래의 단계를 차례대로 수행하여 최종 출력 결과를 텍스트 문서 분석을 위한 정보로서 활용함.

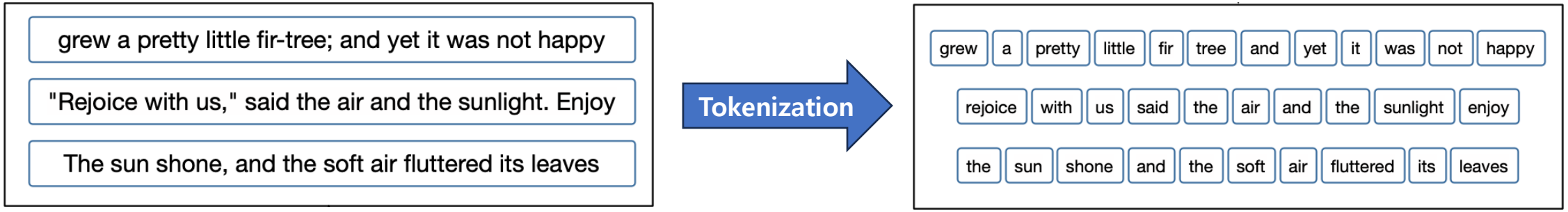
  ① 토큰화 (tokenization)

  ② 불용어 제거 (stop word elimination)

  ③ 원형 변환 (stemming)

① 토큰화 (Tokenization)

- 주어진 문서 내의 텍스트를 최소 의미 단위인 토큰(token)으로 나누는 작업
- 예제



- 가장 간단한 토큰화 방법
  - ➢ 공백(띄어쓰기) 기반으로 토큰화
  - ➢ 단, 실무에서는 의미 정보에 기반하여 보다 정확하게 토큰화를 수행하기 위해 토큰화 알고리즘(tokenization algorithm)을 활용함.

② 불용어 제거(Stop word elimination)

- 불용어(Stop word)란?
  - ➢ 자연어 처리에서 유의미한 정보를 포함하지 않는 단어
  - ➢ (예) 관사, 접속사, 조사, 감탄사, 의성어, … 등
- 주어진 토큰 리스트(token list or token sequence)의 각 토큰에 대하여, 해당 토큰이 가리키는 단어가 불용어인지 확인하고, 불용어에 해당된다면 이를 제거
- 이를 위해, 각 언어 및 응용 별로 불용어 사전(stop word dictionary)이 필요하며, 불용어 사전은 응용 분야별로 다르게 정의될 수 있음.
- 예제
  - ➢ 영어 불용어 리스트: **`from nltk.corpus import stopwords`**
  - ➢ 한글 불용어 리스트: 인터넷에서 다운로드 받아서 사용 가능함.

② 불용어 제거(Stop word elimination)

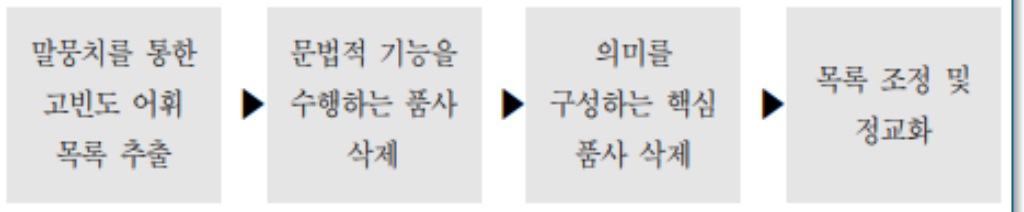
- 불용어 예제

한글 불용어 관련 연구: 텍스트마이닝을 위한 한국어 불용어 목록 연구
(길호현 저, 우리말글, Vol. 78, 우리말글학회, 2018.09.30.)

영어 불용어	한글 불용어
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it's", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further',	가량, 가지, 각, 간, 갖은, 개, 개국, 개년, 개소, 개월, 걔, 거, 거기, 거리, 건, 것, 겨를, 격, 겸, 고, 군, 군데, 권, 그, 그거, 그것, 그곳, 그까짓, 그네, 그녀, 그놈, 그대, 그래, 그래도, 그서, 그러나, 그러니, 그러니까, 그러다가, 그러면, 그러면서, 그러므로, 그러자, 그런, 그런 데, 그럼, 그렇지만, 그루, 그리고, 그리하여, 그분, 그이, 그쪽, 근, 근데, 글쎄, 글쎄요, 기, 김, 나, 나름, 나위, 남짓, 내, 냥, 너, 너희, 네, 네놈, 녀석, 년대, 년도, 누구, 니, 다른, 다만, 단, 달, 편, 저희, 적, 전, 점, 제, 조, 주, 주일, 줄, 중, 즈음, 즉, 지, 지경, 지난, 집, 짝, 쪽, 쯤, 차, 참, 채, 척, 첫, 체, 초, 총, 측, 치, 큰, 킬로미터, 타, 터, 턱, 톤, 통, 투, 판, 퍼센트, 편, 평, 푼, 하기야, 하긴, 하물며, 하지만, 한, 한두, 한편, 허허, 헌, 현, 호, 혹은, 회, 흥

<표 1> 일반 및 전문영역 불용어 리스트의 예(김판구·조유근, 1993:811)

일반 불용어 리스트의 예	전문 영역 불용어 리스트의 예(시사 분야)
가득, 가량, 가려, 가령, 가서, 가운, 가장, 가하, 각기, 각자, 각종, 간간, 간의, 갑자, 갑절, 강한, 거기, 거대, 거두, 거듭, 거의, 거치, 결국, 결코, 계속, 고로, 고루, 고사이, 고작, 관하, 관해 등등	가능, 가능성, 가세, 가시, 가중, 각국, 각급, 각당, 각도, 각지, 간격, 간단, 간소, 간접, 간주, 감소, 감수, 감안, 감퇴, 갑을, 강력, 강세, 강제, 강조, 개인, 거듭, 거래, 거북 등등



<그림 5> 한국어 불용어 목록 선정 절차

③ Stemming and Lemmatization

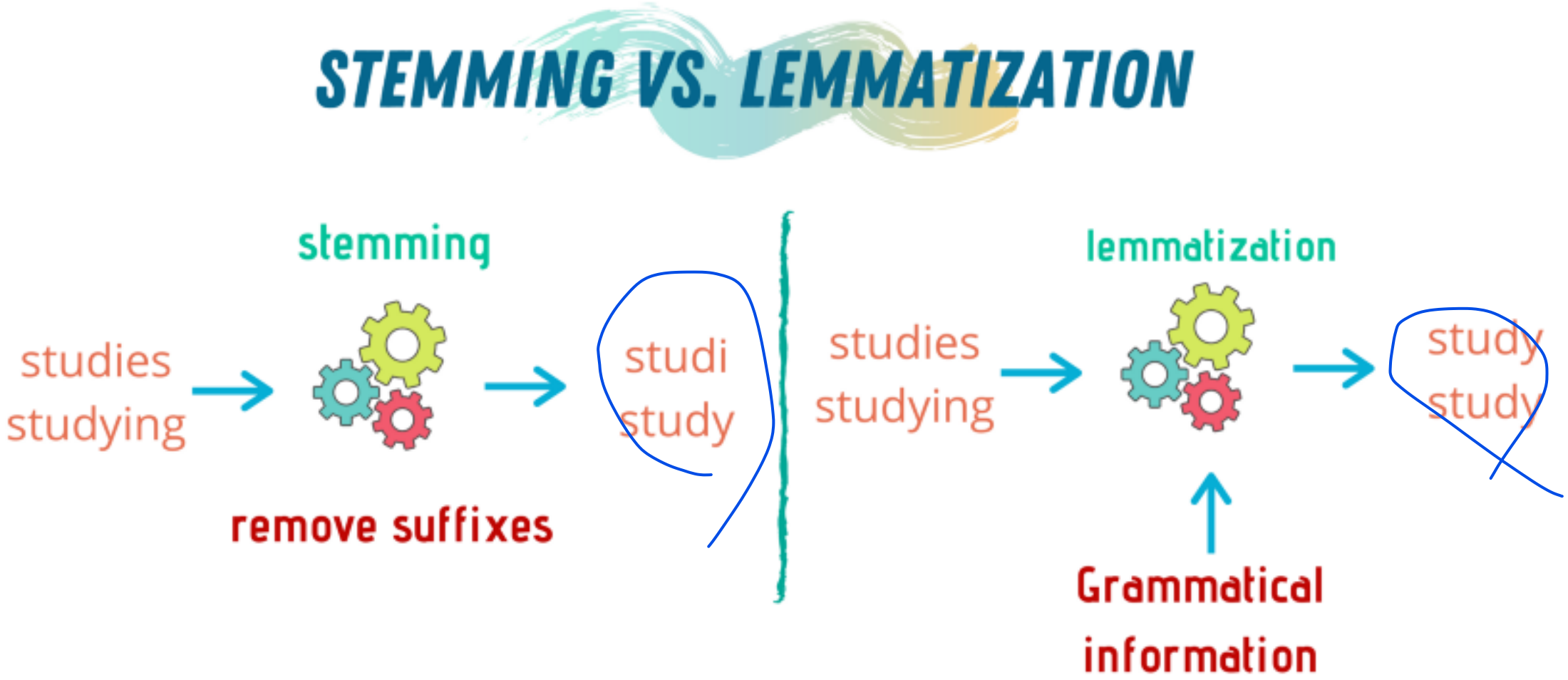
- 주어진 토큰 리스트의 각 토큰에 대하여, 해당 토큰이 가리키는 단어의 변화를 제거하여 일관된 형태로 변환하는 작업
- 예제

- 대표적인 방법
  - Stemming algorithms: Porter, Snowball, Lancaster …
  - Lemmatization algorithms:

③ Stemming and Lemmatization

- **Stemming**
  - 단어의 어간을 추출 작업
    =동사나 형용사 같은 용언이 활용할 때 형태가 변하지 않는 중심 부분
  - 각 언어별로 정해진 규칙에 따라 해당 토큰이 가리키는 단어의 어미를 자르는 역할
  - 단순히 단어의 어미를 제거하는 역할을 수행하므로, stemming된 단어는 사전에 없는 단어일 수도 있으나, 텍스트 분석에는 큰 영향을 미치지 않는 경우가 대부분임
- **Lemmatization**
  - 표제어(단어의 원형) 추출하는 작업
  - 주어진 토큰이 가리키는 단어의 원형으로 변환한다.
  - 이를 위해, 언어별 사전(dictionary)가 필요하며 영어의 경우 WordNet이 널리 활용됨
    =단어들을 대규모 그래프 형태로 구조화 시킨 일종의 사전
  - Stemming과는 달리 lemmatization이 수행된 단어는 사전에 포함된 정상적인 단어이므로 의미 정보를 정확히 유지하고 싶은 경우에 널리 활용됨

③ Stemming and Lemmatization



Reference: https://212digital.medium.com/what-is-lemmatization-and-stemming-in-nlp-e25e142332c4

Ⅱ. TF-IDF 벡터의 기본 개념

주요 개념 요약

- 문서의 분석을 위해서는 모든 문서에 대하여 전처리(preprocessing) 작업을 우선적으로 수행해야 한다.
- 주요 용어
  - **문서(Document)**: 단어들의 집합
  - **코퍼스(Corpus)**: 대규모 문서들의 집합
  - **딕셔너리(Dictionary)**: 코퍼스 내 모든 문서들이 포함하는 단어들의 집합
- 텍스트 문서를 분석하기 위해서는
  - 각각의 문서를 벡터(vector)로 표현해야 한다.
  - 이를 위해, 문서 벡터(document vector)의 각 성분은 어떻게 계산되며, 그것이 의미하는 바가 무엇인지 이해해야 한다.

주어진 문서를 TF-IDF 벡터로 표현하는 절차

① 각 문서에 대한 전처리 작업 수행

② 각 문서에 포함된 단어들을 합집합한 결과인 Dictionary D를 도출

=문서들의 집합(corpus)을 하나의 집합으로 만들어버리기

③ Dictionary D 에 포함된 각 단어들에 대하여 TF-Score 계산

④ Dictionary D 에 포함된 각 단어들에 대하여 IDF-Score 계산

⑤ 각 단어들에 대하여 TF-Score와 IDF-Score를 곱하여 TF-IDF Score를 계산

⑥ 각 문서를 TF-IDF 벡터로 표현

Term Frequency (TF)

- 문서 내에서 해당 단어(term)가 등장하는 횟수(frequency)
- 코퍼스 C 내의 문서 D에 포함된 단어 t에 대한 TF-Score 계산 방법

$$\mathrm{tf}(t, D) = \frac{f_{t,D}}{\sum_{t' \in D} f_{t',D}}$$

문서D에 있는 단어t의 등장횟수

문서D에 포함된 단어 t의 빈도 수의 비율

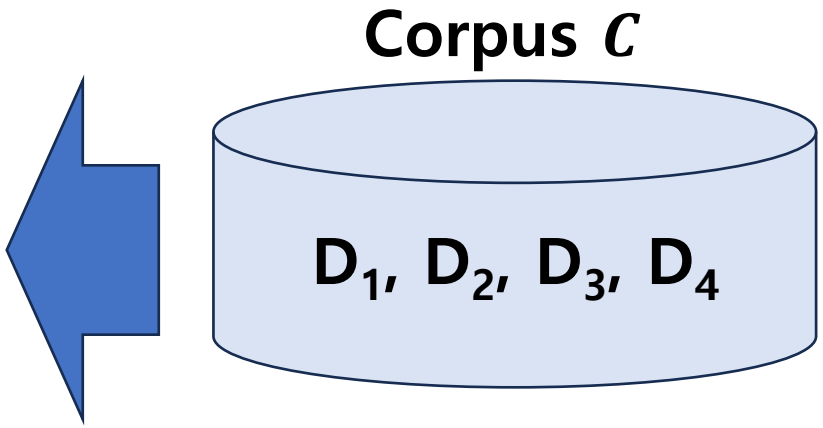
frequency

문서 D에 포함된 모든 단어의 빈도수 합

- 상기 수식의 의미
  - $f_{t,D}$: 문서 D에서 단어 t의 등장 횟수(=term frequency)
  - $t' \in D$: 문서 D에 포함된 임의의 단어 t'
  - $\sum_{t' \in D} f_{t',D}$: 문서 D에 포함된 모든 단어의 빈도수의 합
  - $\mathrm{tf}(t, D)$: 문서 D에 포함된 단어 t의 빈도 수의 비율

Term Frequency (TF)

- 각 문서의 내용
  - D_1: The sky is blue.
  - D_2: The sun is bright today.
  - D_3: The sun in the sky is bright.
  - D_4: We can see the shining sun, the bright sun.



- 문서 D_1~D_4에서 토큰화와 불용어(stop word) 제거, Stemming을 수행한 결과
  - D_1: sky blue
  - D_2: sun bright today
  - D_3: sun sky bright
  - D_4: can see shining sun bright sun

- **코퍼스(Corpus)**
  - ➢ **대규모 텍스트 문서들의 집합**

Term Frequency (TF)

- 각 문서 별 단어(term)로부터 Dictionary D를 구한다.
  - D_1: sky blue
  - D_2: sun bright today
  - D_3: sun sky bright
  - D_4: can see shining sun bright sun

$$\boldsymbol{D = D_1 \cup D_2 \cup D_3 \cup D_4}$$

= {blue, bright, can, see, shining, sky, sun, today}

- Dictionary란, 코퍼스 C에 포함된 모든 문서들이 포함하고 있는 단어(term)들의 집합이다. 즉, dictionary D는 아래와 같다.

$$D = \{\, t \mid t \in D_i \ \text{ s.t. } \forall D_i \in C \,\}$$

Term Frequency (TF)

- 각 문서 내 단어 리스트와 딕셔너리 D
  - D_1: sky blue
  - D_2: sun bright today
  - D_3: sun sky bright
  - D_4: can see shining sun bright sun
  - D = {blue, bright, can, see, shining, sky, sun, today}

$$\mathrm{tf}(t, D) = \frac{f_{t,D}}{\sum_{t' \in D} f_{t',D}}$$

- 상기 딕셔너리 D의 각 term에 대한 TF-Score를 계산하면 다음과 같다.

	blue	bright	can	see	shining	sky	sun	today	시그마
D_1	**1/2**	0	0	0	0	**1/2**	0	0	2
D_2	0	**1/3**	0	0	0	0	**1/3**	**1/3**	3
D_3	0	**1/3**	0	0	0	**1/3**	**1/3**	0	3
D_4	0	**1/6**	**1/6**	**1/6**	**1/6**	0	**1/3**	0	6

Inverse Document Frequency (IDF)

- Document frequency (DF)
  - 단어 t를 포함하는 코퍼스 내 문서(또는 문장)의 개수로 n_t로 표기

- Inverse Document Frequency (IDF)
  - 코퍼스(= C) 내 전체 문서의 개수가 N일 때(즉, $N = |C|$), 단어 t를 포함하는 문서 개수의 역수(IDF)는 다음과 같이 계산한다.

$$\mathrm{idf}(t, C) = \log \frac{N}{n_t}$$

log 문서개수/단어 t 포함 문서개수

Inverse Document Frequency (IDF)

- DF가 아닌 IDF를 사용하는 이유
  - ➢ 모든 문서에 걸쳐서 자주 등장하는 단어 → 문서들을 분류할 때 도움이 되지 않음.
  - ➢ 즉, 이러한 단어들의 경우 문서 분석 시 의미상으로 중요하지 않은 경우가 대부분이므로 이러한 단어에 대한 가중치는 낮추는 것이 더욱 바람직함.

서술형으로 나올듯

- IDF-Score 계산 시 log 함수를 사용하는 이유
  - ➢ 텍스트 코퍼스 내 문서들의 개수는 일반적으로 매우 많다.
  - ➢ 이 경우 IDF-Score의 분자 값이 너무 커져서 TF-score와 함께 사용하기 어렵다.
  - ➢ 이때, log 함수를 사용하면 단조 증가(monotonic increasing)의 속성은 유지하면서도 N의 크기를 줄일 수 있어 TF-Score와 함께 사용할 수 있다.

underflow 현상 방지

원래 이건데

로그 함수는 이래 됨

→ 가중치↓

Inverse Document Frequency (IDF)

- 예제 문서
  - D_1: sky blue
  - D_2: sun bright today
  - D_3: sun sky bright
  - D_4: can see shining sun bright sun
  - D = {blue, bright, can, see, shining, sky, sun, today}

- 상기 딕셔너리 내 각 단어에 대한 IDF-Score를 계산해 보자.

Inverse Document Frequency (IDF)

- 예제:
  - D_1: sky blue
  - D_2: sun bright today
  - D_3: sun sky bright
  - D_4: can see shining sun bright sun
  - D = {blue, bright, can, see, shining, sky, sun, today}
  - $N = |C| = 4$

$$\mathrm{idf}(t, C) = \log \frac{N}{n_t}$$

t	n_t	N/n_t	IDF(t, C)
blue	1	4 / 1	$\log 4 = 0.602$
bright	3	4 / 3	$\log \frac{4}{3} = 0.125$
can	1	4 / 1	$\log 4 = 0.602$
see	1	4 / 1	$\log 4 = 0.602$
shining	1	4 / 1	$\log 4 = 0.602$
sky	2	4 / 2	$\log 2 = 0.301$
sun	3	4 / 3	$\log \frac{4}{3} = 0.125$ =log4 - log3
today	1	4 / 1	$\log 4 = 0.602$

$\frac{1}{3}$ $\log \frac{4}{3}$

TF-IDF Score

- 문서 내 단어 t에 대한 TF-IDF Score는, 해당 단어에 대한 TF-Score와 IDF-Score를 곱한 결과로, 다음과 같이 계산된다.

$$\mathrm{tfidf}(t, D, C) = \boxed{\mathrm{tf}(t, D) \times \mathrm{idf}(t, C)}$$

$$= \frac{f_{t,D}}{\sum_{t' \in D} f_{t',D}} \times \log \frac{N}{n_t}$$

중요제는 나옴

TF-IDF 벡터

- 각 단어에 대한 TF-score와 IDF-score를 곱한 결과
- TF-IDF score는 모든 단어별로 할당되며, 이를 토대로 각 문서(또는 문장)에 대한 TF-IDF vector를 구성할 수 있음.
- 문서를 벡터로 표현하는 방법
  - TF-IDF 벡터의 각 성분은 해당 단어의 TF-IDF Score를 의미한다.
  - 예를 들어, Dictionary D = {blue, bright, can, see, shining, sky, sun, today}일 때, 문서 D_k에 대한 TF-IDF 벡터는 다음과 같다.

$$\overrightarrow{D_k} = [s_{k,1}, s_{k,2}, s_{k,3}, s_{k,4}, s_{k,5}, s_{k,6}, s_{k,7}, s_{k,8}]$$

  - 위 벡터 $\overrightarrow{D_k}$의 성분 $s_{k,i}$는 Dictionary D 집합의 i번째 단어의 TF-IDF Score를 의미한다.

주어진 문서를 TF-IDF 벡터로 표현하기

- 아래의 문서 D_1 ~ D_4를 TF-IDF 벡터 $\overrightarrow{D_1} \sim \overrightarrow{D_4}$로 표현해보자.
  - D_1: sky blue
  - D_2: sun bright today
  - D_3: sun sky bright
  - D_4: can see shining sun bright sun

	blue	bright	can	see	shining	sky	sun	today
D_1	1/2	0	0	0	0	1/2	0	0
D_2	0	1/3	0	0	0	0	1/3	1/3
D_3	0	1/3	0	0	0	1/3	1/3	0
D_4	0	1/6	1/6	1/6	1/6	0	1/3	0
IDF-Score	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

TF-Scores

IDF-Scores

주어진 문서를 TF-IDF 벡터로 표현하기

TF-Scores	blue	bright	can	see	shining	sky	sun	today
D_1	1/2	0	0	0	0	1/2	0	0
D_2	0	1/3	0	0	0	0	1/3	1/3
D_3	0	1/3	0	0	0	1/3	1/3	0
D_4	0	1/6	1/6	1/6	1/6	0	1/3	0

$\times$

IDF-Score	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

- $\overrightarrow{D_1} = [0.301,\ 0,\ 0,\ 0,\ 0,\ 0.151,\ 0,\ 0]$
- $\overrightarrow{D_2} = [0,\ 0.0417,\ 0,\ 0,\ 0,\ 0,\ 0.0417,\ 0.201]$
- $\overrightarrow{D_3} = [0,\ 0.0417,\ 0,\ 0,\ 0,\ 0.100,\ 0.0417,\ 0]$
- $\overrightarrow{D_4} = [0,\ 0.0209,\ 0.100,\ 0.100,\ 0.100,\ 0,\ 0.0417,\ 0]$

Q & A